

Statistical Inference and Modelling - Complete Hands-On Notebooks

BITS Pilani | Course Team

August 25, 2025

Contents

1	Module 1: Basic Probability & Statistics	3
1.1	Section 1.1: Setting up the Environment	3
1.2	Section 1.2: Exploratory Data Analysis (EDA) and Descriptive Statistics	3
1.3	Section 1.3: Basic Probability Concepts	5
1.4	Section 1.4: Random Variables and Distributions	7
1.5	Section 1.5: Parametric vs. Non-Parametric Approaches	9
2	Module 2: Introduction to Statistical Inference	11
2.1	Section 2.1: Sampling Theory	11
2.2	Section 2.2: Point Estimation and Properties of Estimators	12
2.3	Section 2.3: The Central Limit Theorem (CLT)	12
3	Module 3: Estimation and Hypothesis Testing	15
3.1	Section 3.1: Interval Estimation (Confidence Intervals)	15
3.2	Section 3.2: Determining Sample Size	16
3.3	Section 3.3: One-Sample Hypothesis Tests	16
4	Module 4: Two-Sample Tests & Categorical Data	18
4.1	Section 4.1: Inferences for Two Population Means	18
4.2	Section 4.2: Two-Sample Proportion Test (A/B Testing)	18
4.3	Section 4.3: Test of Independence (Chi-Square Test)	19
5	Module 5: Analysis of Variance (ANOVA)	20
5.1	Section 5.1: Performing One-Way ANOVA	20
5.2	Section 5.2: Post-Hoc Testing (Tukey's HSD)	21
6	Module 6: Simple Linear Regression (SLR)	23
6.1	Section 6.1: Building the Regression Model	23
6.2	Section 6.2: Assessing the Model and Residuals	24
7	Module 7: Multiple Regression	26
7.1	Section 7.1: Building the Multiple Regression Model	26
7.2	Section 7.2: Multicollinearity (VIF)	27
8	Module 8: Forecasting & Time Series Analysis	28
8.1	Section 8.1: Time Series Decomposition	28
8.2	Section 8.2: Holt's Linear Trend Model (Double Exponential Smoothing)	28
9	Module 9: Factor Analysis	31

9.1	Section 9.1: Performing Factor Analysis	31
9.2	Section 9.2: Interpreting Factor Loadings	32
10	Module 10: Cluster Analysis	33
10.1	Section 10.1: K-Means Clustering	33
11	Module 11: Maximum Likelihood Methods	36
11.1	Section 11.1: Visualizing the Likelihood Function	36
11.2	Section 11.2: Finding the MLE for a Normal Distribution	37
12	Module 12: Discrete Response Models	39
12.1	Section 12.1: Building the Logistic Regression Model	39
12.2	Section 12.2: Interpreting the Model and Odds Ratios	39

1 Module 1: Basic Probability & Statistics

Welcome to the first hands-on notebook for Statistical Inference and Modelling! This module revisits fundamental concepts of probability, random variables, and probability distributions. We will use Python to explore these concepts practically.

Learning Objectives:

- Perform exploratory data analysis (EDA) with descriptive statistics.
- Understand and simulate basic probability concepts.
- Explore key random variables and their distributions (Binomial, Poisson, Normal).
- Differentiate between parametric and non-parametric approaches (Conceptual).

1.1 Section 1.1: Setting up the Environment

We begin by importing the necessary Python libraries. These tools are essential for data manipulation, numerical computation, and visualization.

```
# Import necessary libraries
import numpy as np # For numerical operations
import pandas as pd # For data manipulation and analysis
import matplotlib.pyplot as plt # For plotting
import seaborn as sns # For statistical data visualization
from scipy import stats # For statistical functions
import math # For mathematical operations like sqrt

# Setting style for visualizations
sns.set_style("whitegrid")
# Setting a global seed for reproducibility across the notebooks
GLOBAL_SEED = 42
np.random.seed(GLOBAL_SEED)

print("Libraries imported successfully!")
```

```
## Libraries imported successfully!
```

1.2 Section 1.2: Exploratory Data Analysis (EDA) and Descriptive Statistics

Descriptive statistics summarize the main features of a dataset. Let's start with the fundamental concepts of **Population** and **Sample**.

- **Population:** The entire group that you want to draw conclusions about.
- **Sample:** A specific subset of the population from which you collect data.

```
# Let's simulate a population of 50,000 individuals and their annual incomes.
# Income data is often skewed, so we use a gamma distribution to model it.
population_incomes = np.random.gamma(shape=5, scale=10000, size=50000)

print(f"Population size: {len(population_incomes)}")
```

```
## Population size: 50000
```

```
print(f"Population Mean Income ( ): ${np.mean(population_incomes):.2f}")
```

```
## Population Mean Income ( ): $50087.50
```

```
# Now, let's draw a random sample of 200 individuals from this population.
sample_incomes = np.random.choice(population_incomes, size=200, replace=False)
```

```
print(f"\nSample size: {len(sample_incomes)}")

##
## Sample size: 200
print(f"Sample Mean Income ( $\bar{x}$ ): ${np.mean(sample_incomes):.2f}")

## Sample Mean Income ( $\bar{x}$ ): $49741.70
```

1.2.1 1.2.1 Measures of Central Tendency (Mean, Median, Mode)

```
# Calculate the Mean (Average)
sample_mean = np.mean(sample_incomes)
print(f"Mean Income: ${sample_mean:.2f}")

## Mean Income: $49741.70

# Calculate the Median (The middle value when data is sorted)
sample_median = np.median(sample_incomes)
print(f"Median Income: ${sample_median:.2f}")

## Median Income: $46978.97

# Calculate the Mode (The most frequent value)
favorite_colors = np.random.choice(['Red', 'Blue', 'Green', 'Red', 'Blue', 'Blue', 'Yellow'], 50)
# FIX: Use pandas to find the mode for categorical (non-numeric) data.
mode_result = pd.Series(favorite_colors).mode()[0]
print(f"\nMode of Favorite Colors: {mode_result}")

##
## Mode of Favorite Colors: Blue
```

1.2.2 1.2.2 Measures of Dispersion (Variance, Standard Deviation, Range)

```
# Range (Difference between maximum and minimum)
sample_range = np.max(sample_incomes) - np.min(sample_incomes)
print(f"Range of Incomes: ${sample_range:.2f}")

## Range of Incomes: $102053.08

# Variance (Average of squared differences from the Mean, with Bessel's correction)
sample_variance = np.var(sample_incomes, ddof=1)
print(f"Variance of Incomes: {sample_variance:.2f}")

## Variance of Incomes: 434341208.18

# Standard Deviation (Square root of the Variance)
sample_std = np.std(sample_incomes, ddof=1)
print(f"Standard Deviation of Incomes: ${sample_std:.2f}")

## Standard Deviation of Incomes: $20840.85
```

1.2.3 1.2.3 Visualizing Data

```
plt.figure(figsize=(10, 6))

# Histogram with a Kernel Density Estimate (KDE)
```

```
sns.histplot(sample_incomes, kde=True, bins=20)
plt.title('Distribution of Sample Incomes')
plt.xlabel('Income ($)')
plt.ylabel('Frequency')

# Adding Mean and Median lines to visualize the skewness
plt.axvline(sample_mean, color='r', linestyle='--', label=f'Mean (${sample_mean:.2f})')
plt.axvline(sample_median, color='g', linestyle='-', label=f'Median (${sample_median:.2f})')
plt.legend()
plt.show()
```

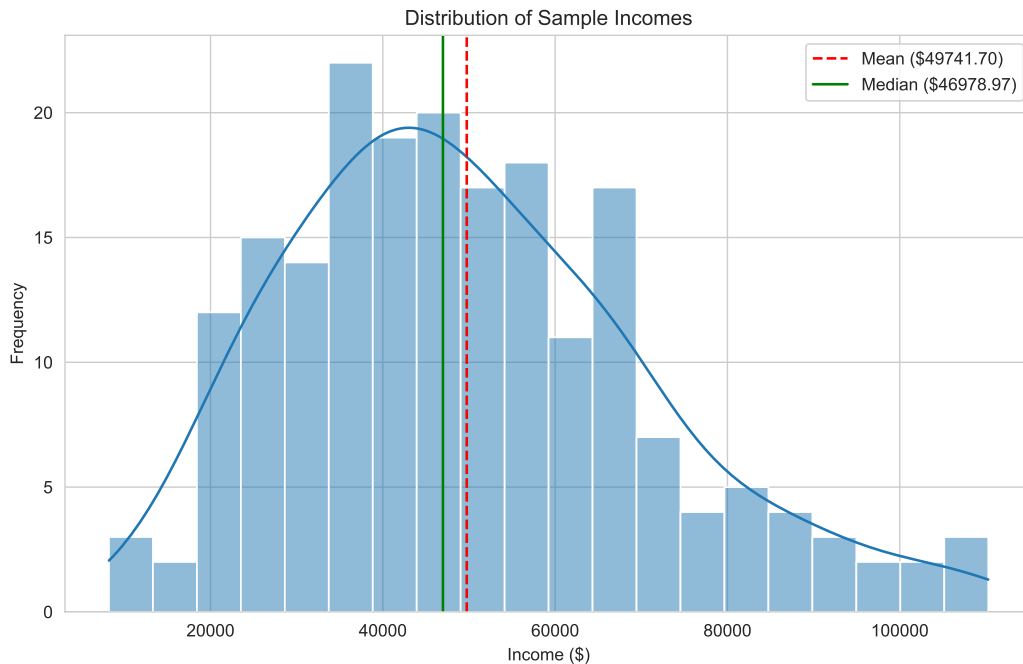


Figure 1: Distribution of Sample Incomes, showing right-skew.

```
# Box Plot (Box-and-Whisker Plot): Shows the quartiles and potential outliers.
plt.figure(figsize=(8, 5))
sns.boxplot(x=sample_incomes)
plt.title('Box Plot of Sample Incomes')
plt.xlabel('Income ($)')
plt.show()
```

1.3 Section 1.3: Basic Probability Concepts

1.3.1 1.3.1 Simulating Experiments and the Law of Large Numbers

The Law of Large Numbers states that as the number of trials increases, the empirical probability approaches the theoretical probability.

```
# Simulating 10 coin flips (0=Tails, 1=Heads)
flips = np.random.randint(0, 2, size=10)
print(f"10 Flips: {flips}")
```

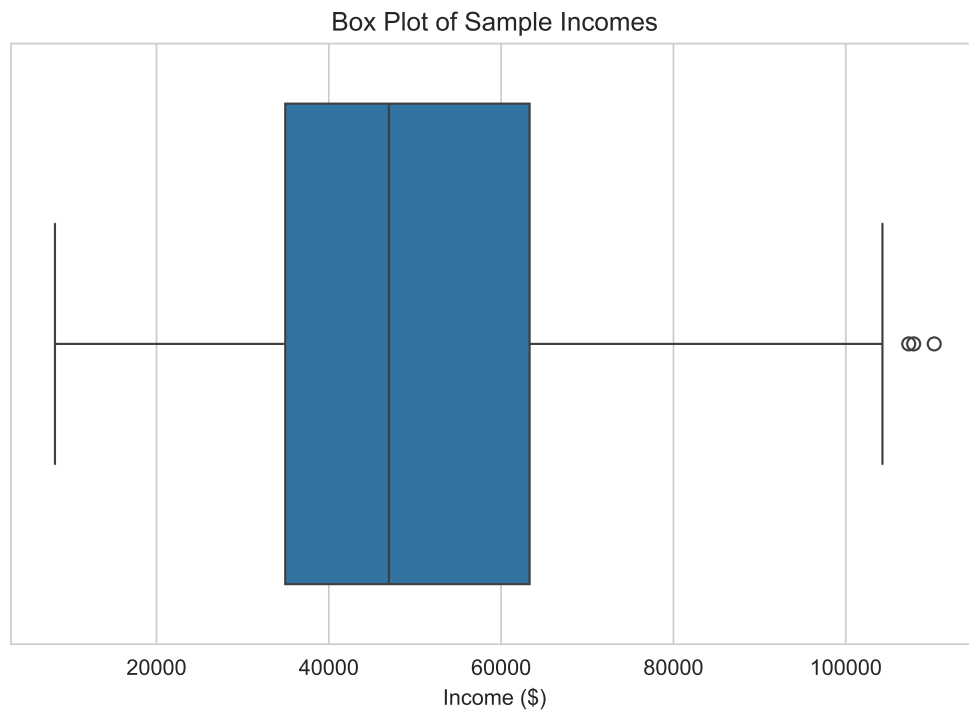


Figure 2: Box Plot of Sample Incomes

```
## 10 Flips: [1 0 0 0 0 0 0 0 0 1]
print(f"Empirical Probability of Heads (10 flips): {np.sum(flips)/10:.4f}")

## Empirical Probability of Heads (10 flips): 0.2000
# Simulating 10000 coin flips
many_flips = np.random.randint(0, 2, size=10000)
empirical_prob_heads = np.sum(many_flips) / 10000

print(f"\nEmpirical Probability of Heads (10000 flips): {empirical_prob_heads:.4f}")

##
## Empirical Probability of Heads (10000 flips): 0.5158
```

1.3.2 1.3.2 Conditional Probability

Example: The relationship between Smoking (S) and Lung Disease (L).

```
# Data from a contingency table
Total = 100
P_S = 40 / Total # P(Smoker)
P_L_and_S = 15 / Total # P(Lung Disease AND Smoker)

# Question: What is the probability of having Lung Disease GIVEN the person is a Smoker? P(L|S)
P_L_given_S = P_L_and_S / P_S
```

```
print(f"P(S): {P_S:.3f}")

## P(S): 0.400

print(f"P(L|S): {P_L_given_S:.3f}")

## P(L|S): 0.375
```

1.4 Section 1.4: Random Variables and Distributions

1.4.1 1.4.1 Discrete Distribution: Binomial Distribution

Models the number of successes in a fixed number of independent trials.

```
n = 10
p = 0.5

# Probability of exactly 6 heads (k=6) using the Probability Mass Function (PMF)
prob_6_heads = stats.binom.pmf(k=6, n=n, p=p)
print(f"P(X=6): {prob_6_heads:.4f}")
```

```
## P(X=6): 0.2051

# Probability of 6 or fewer heads (k<=6) using the Cumulative Distribution Function (CDF)
prob_6_or_fewer = stats.binom.cdf(k=6, n=n, p=p)
print(f"P(X<=6): {prob_6_or_fewer:.4f}")
```

```
## P(X<=6): 0.8281

# Visualizing the Binomial Distribution
k_values = np.arange(0, n + 1)
pmf_values = stats.binom.pmf(k_values, n=n, p=p)

plt.figure(figsize=(8, 4))
plt.bar(k_values, pmf_values)
plt.title('Binomial Distribution (n=10, p=0.5)')
plt.xlabel('Number of Heads (k)')
plt.ylabel('Probability (PMF)')
plt.xticks(k_values)
```

```
## ([<matplotlib.axis.XTick object at 0x160502a60>, <matplotlib.axis.XTick object at 0x160502250>, <matplotlib.axis.XTick object at 0x160502250>], [  
plt.show()
```

1.4.2 1.4.2 Discrete Distribution: Poisson Distribution

Models the number of events occurring in a fixed interval of time or space.

```
# Example: A call center receives an average of 5 calls per hour (lambda=5).
lam = 5

# Probability of exactly 3 calls in the next hour?
prob_3_calls = stats.poisson.pmf(k=3, mu=lam)
print(f"P(X=3): {prob_3_calls:.4f}")
```

```
## P(X=3): 0.1404

# Visualizing the Poisson Distribution
k_values = np.arange(0, 15)
```

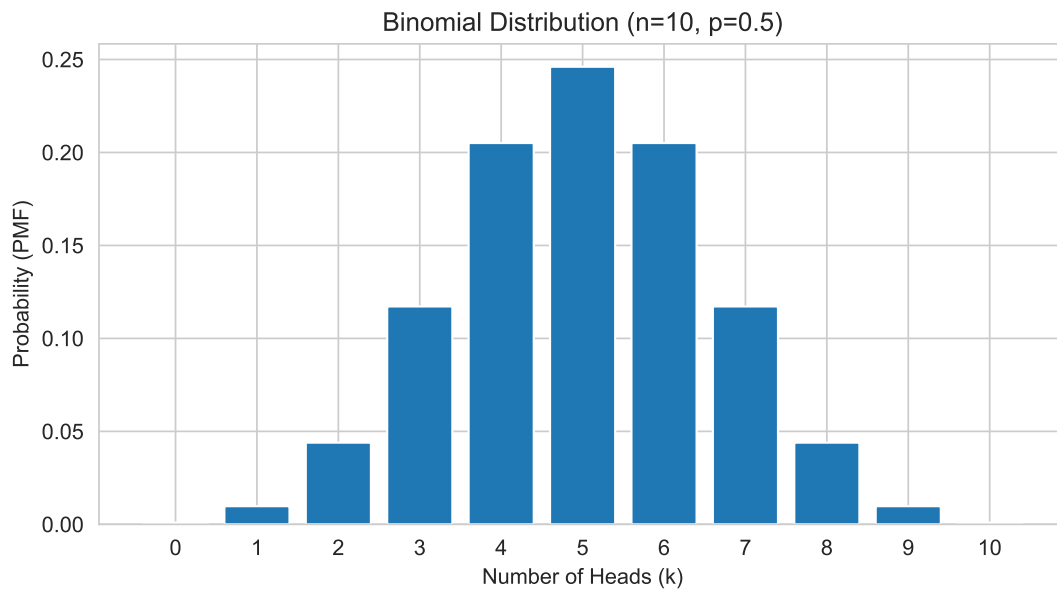


Figure 3: Binomial Distribution PMF for $n=10$, $p=0.5$

```
pmf_values = stats.poisson.pmf(k_values, mu=lam)

plt.figure(figsize=(8, 4))
plt.bar(k_values, pmf_values)
plt.title('Poisson Distribution ( =5)')
plt.xlabel('Number of Calls (k)')
plt.ylabel('PMF')
plt.show()
```

1.4.3 Continuous Distribution: Normal Distribution

The ‘bell curve’, crucial due to the Central Limit Theorem.

Example: Exam scores are normally distributed with mean 70 and std dev 10.

```
mu = 70
```

```
sigma = 10
```

Probability that a student scores less than 60?

```
prob_less_60 = stats.norm.cdf(x=60, loc=mu, scale=sigma)
```

```
print(f"P(X < 60): {prob_less_60:.4f}")
```

```
## P(X < 60): 0.1587
```

Probability that a student scores more than 90?

```
prob_more_90 = 1 - stats.norm.cdf(x=90, loc=mu, scale=sigma)
```

```
print(f"P(X > 90): {prob_more_90:.4f}")
```

```
## P(X > 90): 0.0228
```

Visualizing the Normal Distribution (Probability Density Function - PDF)

```
x_values = np.linspace(mu - 4*sigma, mu + 4*sigma, 1000)
```

```
pdf_values = stats.norm.pdf(x_values, loc=mu, scale=sigma)
```

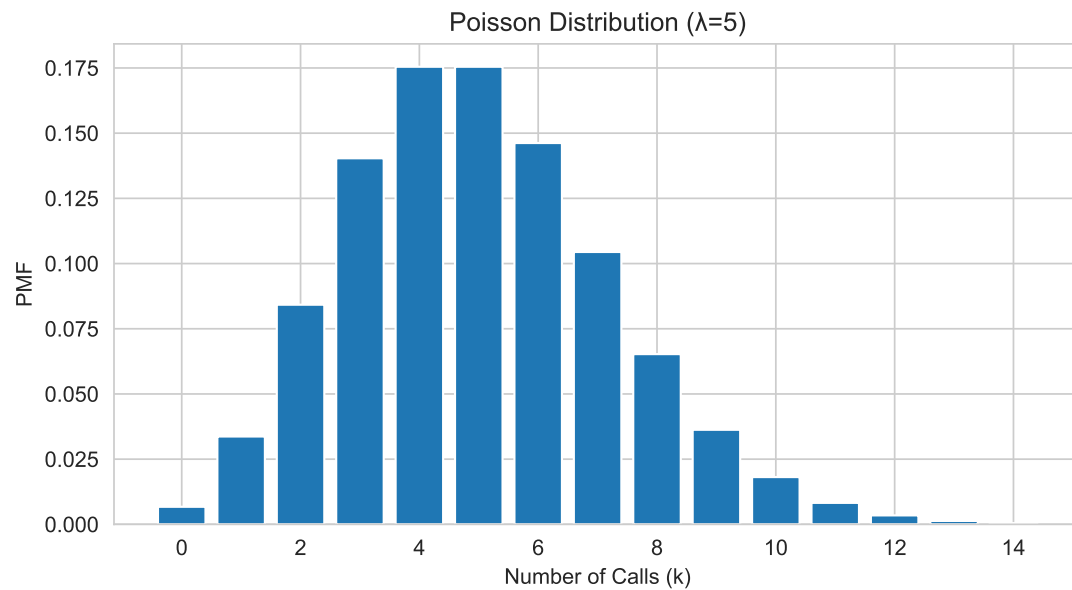


Figure 4: Poisson Distribution PMF for $\lambda=5$

```
plt.figure(figsize=(8, 4))
plt.plot(x_values, pdf_values)
plt.title('Normal Distribution ( $\mu=70$ ,  $\sigma=10$ )')
plt.xlabel('Score')
plt.ylabel('PDF')
# Shading the area for  $P(X > 90)$ 
x_fill = np.linspace(90, mu + 4*sigma, 100)
y_fill = stats.norm.pdf(x_fill, loc=mu, scale=sigma)
plt.fill_between(x_fill, y_fill, color='red', alpha=0.5, label='P(X > 90)')
plt.legend()
plt.show()
```

1.5 Section 1.5: Parametric vs. Non-Parametric Approaches

- **Parametric Methods:** Assume the data follows a specific distribution (e.g., Normal).
- **Non-Parametric Methods:** Do not rely on specific distributional assumptions (distribution-free).

```
# Example: Comparing the center of a small, skewed sample
skewed_data = np.random.exponential(scale=10, size=20)
```

```
# Parametric approach focuses on the Mean
print(f"Mean (Parametric focus): {np.mean(skewed_data):.2f}")
```

```
## Mean (Parametric focus): 9.23
```

```
# Non-Parametric approach focuses on the Median
print(f"Median (Non-Parametric focus): {np.median(skewed_data):.2f}")
```

```
## Median (Non-Parametric focus): 8.15
```

```
# Parametric test (t-test - might be unreliable here)
t_stat, p_val_para = stats.ttest_1samp(skewed_data, 10)
```

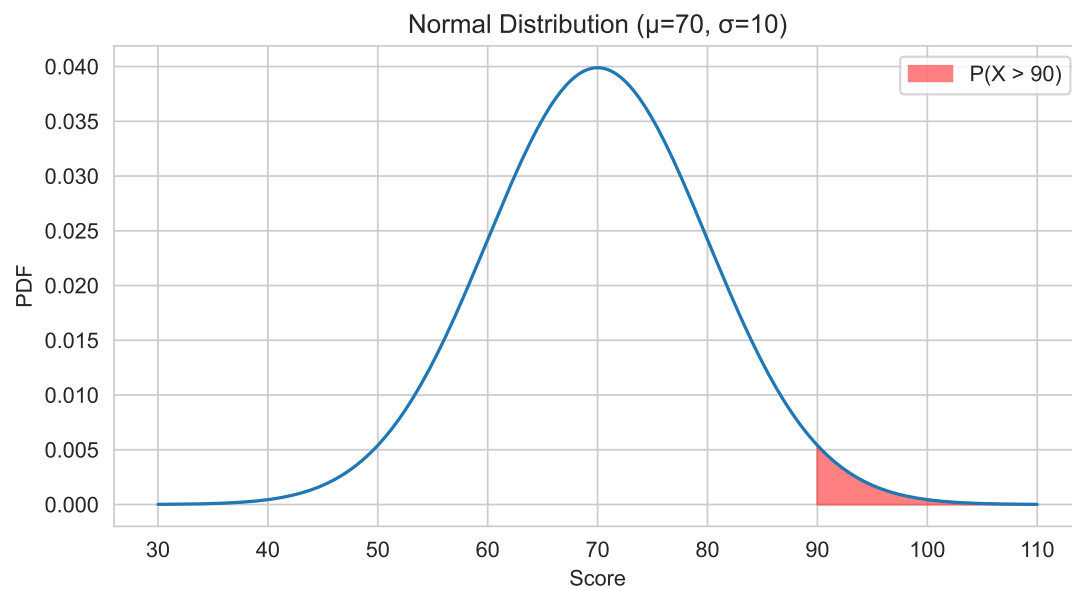


Figure 5: Normal Distribution PDF for $\mu=70, \sigma=10$

```
print(f"\nT-test p-value (Parametric): {p_val_para:.4f}")

##
## T-test p-value (Parametric): 0.6506
# Non-Parametric test (Wilcoxon signed-rank test - tests the median)
try:
    w_stat, p_val_nonpara = stats.wilcoxon(skewed_data - 10)
    print(f"Wilcoxon test p-value (Non-Parametric): {p_val_nonpara:.4f}")
except ValueError as e:
    print(f"Wilcoxon test could not be performed: {e}")

## Wilcoxon test p-value (Non-Parametric): 0.3683
```

2 Module 2: Introduction to Statistical Inference

This module explores how we use samples to make generalizations about populations.

Learning Objectives:

- Understand and apply various sampling methods.
- Define and calculate point estimates and understand their properties.
- Visualize and understand the Central Limit Theorem (CLT).

```
# Resetting seed for this module
np.random.seed(GLOBAL_SEED)
print("Setup Complete")
```

```
## Setup Complete
```

2.1 Section 2.1: Sampling Theory

Let's create a hypothetical university population dataset.

```
# Create a diverse population dataset
data = []
data.extend([{'Faculty': 'Arts', 'GPA': np.random.normal(2.8, 0.5)} for _ in range(1000)])
data.extend([{'Faculty': 'Science', 'GPA': np.random.normal(3.2, 0.4)} for _ in range(800)])
data.extend([{'Faculty': 'Engineering', 'GPA': np.random.normal(3.5, 0.3)} for _ in range(500)])

population_df = pd.DataFrame(data)
population_df['GPA'] = population_df['GPA'].apply(lambda x: max(0.0, min(x, 4.0)))
population_mean_gpa = population_df['GPA'].mean()

print(f"Total Population Size: {len(population_df)}")

## Total Population Size: 2300
print(f"Population Mean GPA (True Parameter ): {population_mean_gpa:.4f}")
```

```
## Population Mean GPA (True Parameter ): 3.1029
```

2.1.1 2.1.1 Simple Random Sampling (SRS)

Every member has an equal chance of selection.

```
sample_size = 100
srs_sample = population_df.sample(n=sample_size, random_state=1)
srs_mean_gpa = srs_sample['GPA'].mean()
print(f"SRS Sample Mean GPA (Estimate  $\bar{x}$ ): {srs_mean_gpa:.4f}")
```

```
## SRS Sample Mean GPA (Estimate  $\bar{x}$ ): 3.1941
```

2.1.2 2.1.2 Stratified Sampling

Ensures the sample structure mirrors the population structure.

```
def stratified_sampler(x):
    fraction = sample_size / len(population_df)
    n_samples = max(1, int(round(len(x) * fraction)))
    if len(x) < n_samples: n_samples = len(x)
    return x.sample(n=n_samples, random_state=1)
```

```
stratified_sample = population_df.groupby('Faculty', group_keys=False).apply(stratified_sampler)
stratified_mean_gpa = stratified_sample['GPA'].mean()
print(f"Stratified Sample Mean GPA (Estimate  $\bar{x}$ ): {stratified_mean_gpa:.4f}")
```

```
## Stratified Sample Mean GPA (Estimate  $\bar{x}$ ): 3.0636
```

2.2 Section 2.2: Point Estimation and Properties of Estimators

A point estimate is a single value used to estimate a population parameter. Good estimators are **unbiased** and **efficient**.

```
# Demonstrating Unbiasedness of the Sample Mean
```

```
sample_means = []
```

```
num_simulations = 5000
```

```
for _ in range(num_simulations):
```

```
    sample = population_df['GPA'].sample(n=50)
```

```
    sample_means.append(sample.mean())
```

```
average_of_sample_means = np.mean(sample_means)
```

```
print(f"True Population Mean ( ): {population_mean_gpa:.4f}")
```

```
## True Population Mean ( ): 3.1029
```

```
print(f"Average of 5000 Sample Means ( $E[\bar{x}]$ ): {average_of_sample_means:.4f}")
```

```
## Average of 5000 Sample Means ( $E[\bar{x}]$ ): 3.1012
```

2.3 Section 2.3: The Central Limit Theorem (CLT)

The sampling distribution of the sample mean will approach a normal distribution as the sample size (n) increases, regardless of the population's distribution.

```
# Create an Exponential population (Highly skewed)
```

```
exponential_population = np.random.exponential(scale=10, size=10000)
```

```
plt.figure(figsize=(8, 4))
```

```
## <Figure size 800x400 with 0 Axes>
```

```
sns.histplot(exponential_population, bins=50)
```

```
## <Axes: ylabel='Count'>
```

```
plt.title('Distribution of the Exponential Population (Not Normal)')
```

```
## Text(0.5, 1.0, 'Distribution of the Exponential Population (Not Normal)')
```

```
plt.show()
```

```
# Define the visualization function
```

```
def visualize_clt(population, sample_size, num_samples=1000):
```

```
    sample_means = [np.mean(np.random.choice(population, size=sample_size)) for _ in range(num_samples)]
```

```
    plt.figure(figsize=(8, 4))
```

```
    sns.histplot(sample_means, kde=True)
```

```
    plt.title(f'Sampling Distribution of the Mean (n={sample_size})')
```

```
    plt.xlabel('Sample Mean')
```

```
    plt.axvline(np.mean(population), color='red', linestyle='--', label='Population Mean')
```

```
    plt.legend()
```

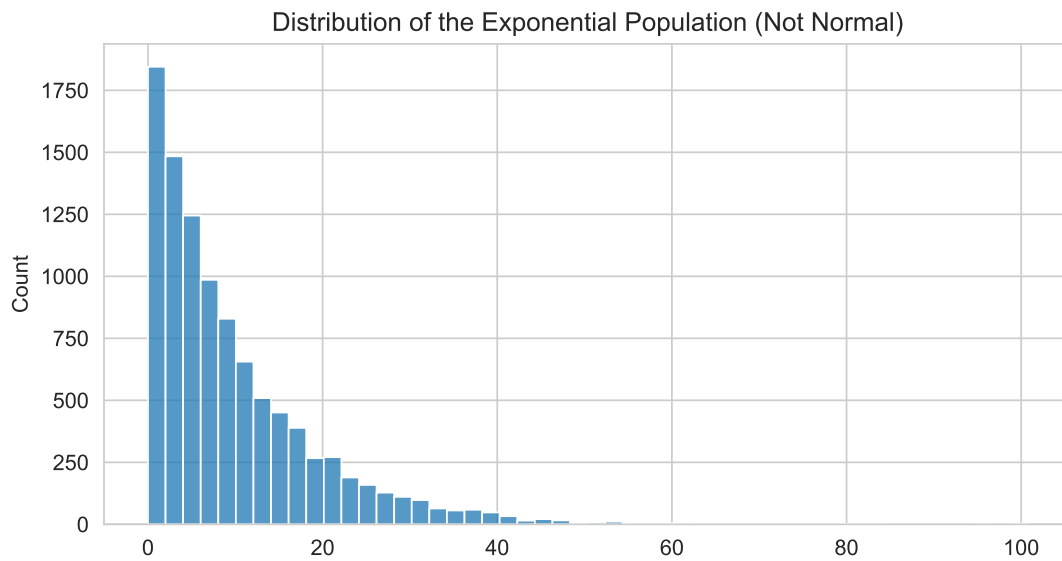


Figure 6: A highly skewed (Exponential) population distribution.

```
plt.show()

# Visualize for n=2 (still skewed), n=10 (less skewed), and n=50 (normal)
visualize_clt(exponential_population, sample_size=2)

visualize_clt(exponential_population, sample_size=10)

visualize_clt(exponential_population, sample_size=50)
```

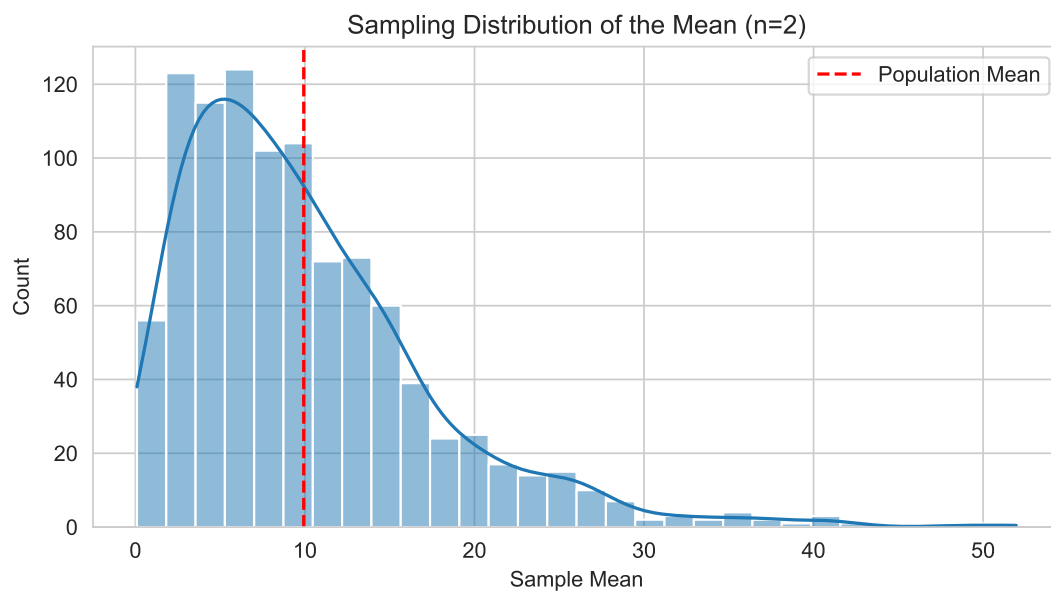


Figure 7: Sampling distribution with $n=50$, approaching normality.

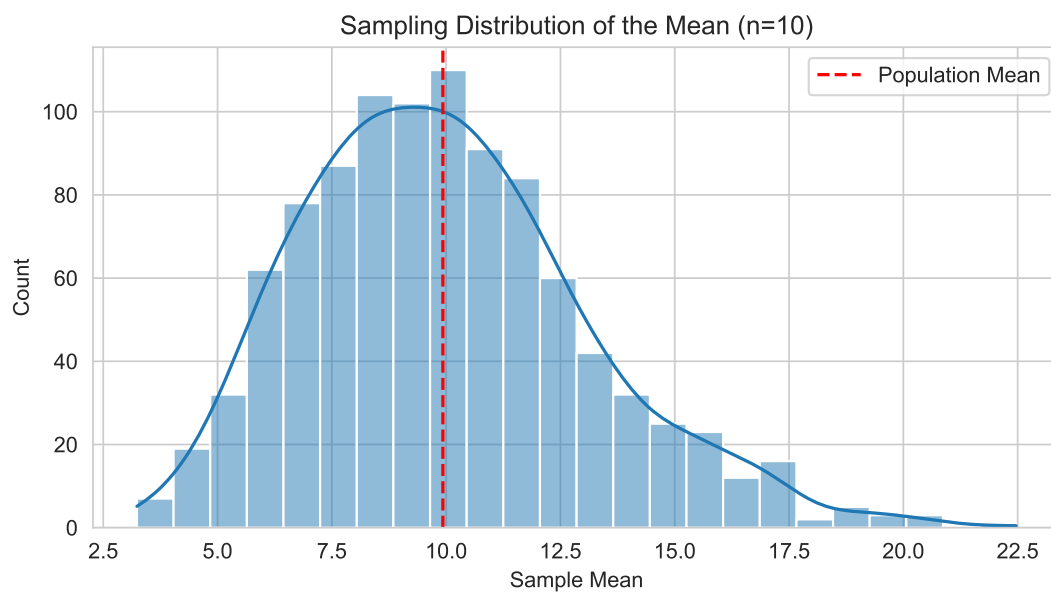


Figure 8: Sampling distribution with $n=50$, approaching normality.

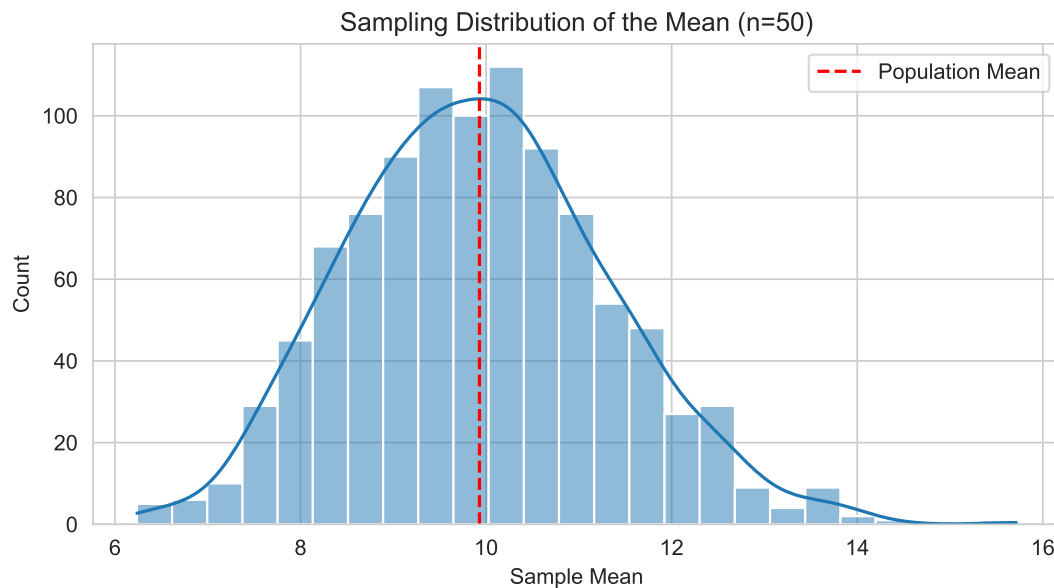


Figure 9: Sampling distribution with $n=50$, approaching normality.

3 Module 3: Estimation and Hypothesis Testing

This module covers Confidence Intervals and the framework of hypothesis testing.

Learning Objectives:

- Construct and interpret Confidence Intervals.
- Calculate the required sample size.
- Understand and conduct one-sample hypothesis tests.

```
# Resetting seed for this module
np.random.seed(GLOBAL_SEED)
print("Setup Complete")
```

```
## Setup Complete
```

3.1 Section 3.1: Interval Estimation (Confidence Intervals)

A Confidence Interval (CI) provides a range of plausible values for a parameter. **CI = Point Estimate $\pm$ Margin of Error.**

3.1.1 3.1.1 Confidence Interval for Mean (known - Z-interval)

```
# Scenario: Light bulb lifespan. =100 hours (known). Sample n=40,  $\bar{x}$ =780 hours.
sigma = 100
n = 40
x_bar = 780
confidence_level = 0.95
alpha = 1 - confidence_level

z_critical = stats.norm.ppf(1 - alpha/2)
standard_error = sigma / math.sqrt(n)
margin_of_error = z_critical * standard_error
```

```

ci_lower = x_bar - margin_of_error
ci_upper = x_bar + margin_of_error

print(f"95% Confidence Interval (Z): ({ci_lower:.2f}, {ci_upper:.2f})")

## 95% Confidence Interval (Z): (749.01, 810.99)

```

3.1.2 3.1.2 Confidence Interval for Mean (unknown - t-interval)

When σ is unknown, we estimate it with the sample standard deviation (s) and use the t-distribution.

```

# Scenario: Student heights. n=20, x̄=170cm, s=8cm.
n = 20
x_bar = 170
s = 8
df = n - 1

t_critical = stats.t.ppf(1 - alpha/2, df=df)
standard_error = s / math.sqrt(n)
margin_of_error = t_critical * standard_error
ci_lower = x_bar - margin_of_error
ci_upper = x_bar + margin_of_error

print(f"95% Confidence Interval (t): ({ci_lower:.2f}, {ci_upper:.2f})")

## 95% Confidence Interval (t): (166.26, 173.74)

```

3.2 Section 3.2: Determining Sample Size

We can calculate the sample size needed for a desired Margin of Error (ME). Formula:

$$n = \left(\frac{Z_{\alpha/2} \cdot \sigma}{ME} \right)^2$$

```

# Scenario: Estimate smartphone battery life. We want ME=0.5 hours with 99% confidence.
# We estimate σ=2 hours.
ME_desired = 0.5
sigma_est = 2
confidence_level = 0.99
alpha = 1 - confidence_level

z_critical = stats.norm.ppf(1 - alpha/2)
n_required = ((z_critical * sigma_est) / ME_desired)**2

print(f"Required sample size (rounded up): {math.ceil(n_required)}")

## Required sample size (rounded up): 107

```

3.3 Section 3.3: One-Sample Hypothesis Tests

3.3.1 3.3.1 One-Sample t-test

Used to compare a sample mean to a hypothesized population mean when σ is unknown.

```

# Scenario: A university claims the average GPA is 3.5. We take a sample of 50 graduates.
# H0: μ = 3.5

```

```

# H:    3.5 (Two-tailed test)
alpha = 0.05
mu_hypothesized = 3.5
sample_data = np.random.normal(loc=3.4, scale=0.4, size=50)

t_statistic, p_value = stats.ttest_1samp(a=sample_data, popmean=mu_hypothesized)

print(f"Sample Mean GPA: {np.mean(sample_data):.4f}")

## Sample Mean GPA: 3.3098
print(f"T-statistic: {t_statistic:.4f}")

## T-statistic: -3.6010
print(f"P-value: {p_value:.4f}")

## P-value: 0.0007
if p_value < alpha:
    print("\nDecision: Reject H . Evidence suggests the average GPA is different from 3.5.")
else:
    print("\nDecision: Fail to reject H . Insufficient evidence.")

##
## Decision: Reject H . Evidence suggests the average GPA is different from 3.5.

```

4 Module 4: Two-Sample Tests & Categorical Data

This module extends hypothesis testing to comparing two populations and analyzing categorical data.

Learning Objectives:

- Perform Independent Samples t-tests.
- Conduct tests for population proportions.
- Conduct a Chi-Square test for independence.

```
from statsmodels.stats.proportion import proportions_ztest
np.random.seed(GLOBAL_SEED)
alpha = 0.05
print("Setup Complete")
```

```
## Setup Complete
```

4.1 Section 4.1: Inferences for Two Population Means

The **Independent Samples t-test** compares the means of two independent groups. A key assumption is the equality of variances, which we check with **Levene's Test**.

```
# Scenario: Comparing two teaching methods.
```

```
method_A_scores = np.random.normal(loc=75, scale=10, size=30)
```

```
method_B_scores = np.random.normal(loc=80, scale=10, size=30)
```

```
# Levene's Test for equal variances
```

```
stat_levene, p_value_levene = stats.levene(method_A_scores, method_B_scores)
```

```
print(f"Levene's Test P-value: {p_value_levene:.4f}")
```

```
## Levene's Test P-value: 0.6223
```

```
equal_var_assumption = p_value_levene >= alpha
```

```
# Perform the t-test, using Welch's t-test if variances are unequal
```

```
t_statistic, p_value = stats.ttest_ind(method_A_scores, method_B_scores, equal_var=equal_var_assumption)
```

```
print(f"T-test P-value: {p_value:.4f}")
```

```
## T-test P-value: 0.0197
```

```
if p_value < alpha:
```

```
    print("\nDecision: Reject H . There is a significant difference between the means.")
```

```
else:
```

```
    print("\nDecision: Fail to reject H.")
```

```
##
```

```
## Decision: Reject H . There is a significant difference between the means.
```

4.2 Section 4.2: Two-Sample Proportion Test (A/B Testing)

Used to compare the proportions of two independent groups.

```
# Scenario: A/B Testing for Website Conversion
```

```
# Layout A: 1000 visitors, 150 converted.
```

```
# Layout B: 1000 visitors, 185 converted.
```

```
# H: p_A >= p_B (Layout B is not better)
```

```
# H: p_A < p_B (Layout B is better)
```

```
n_A, n_B = 1000, 1000
```

```
success_A, success_B = 150, 185
```

```

counts = np.array([success_A, success_B])
nobs = np.array([n_A, n_B])

z_stat, p_value = proportions_ztest(count=counts, nobs=nobs, alternative='smaller')
print(f"P-value: {p_value:.4f}")

## P-value: 0.0180

if p_value < alpha:
    print("\nDecision: Reject H . Layout B has a significantly higher conversion rate.")
else:
    print("\nDecision: Fail to reject H .")

##
## Decision: Reject H . Layout B has a significantly higher conversion rate.

```

4.3 Section 4.3: Test of Independence (Chi-Square Test)

Used to determine if there is a significant relationship between two categorical variables.

```

# Scenario: Relationship between 'Education Level' and 'Preferred Music Genre'.
data = {'High School': [30, 20, 50], 'Bachelors': [35, 40, 25], 'Masters': [25, 45, 10]}
contingency_table = pd.DataFrame(data, index=['Pop', 'Rock', 'Classical'])
print("Contingency Table (Observed Frequencies):\n", contingency_table)

```

```

## Contingency Table (Observed Frequencies):
##           High School  Bachelors  Masters
## Pop                30         35      25
## Rock               20         40      45
## Classical          50         25      10

```

```

chi2_stat, p_value, dof, expected_freq = stats.chi2_contingency(contingency_table)
print(f"\nP-value: {p_value:.4f}")

```

```

##
## P-value: 0.0000

if p_value < alpha:
    print("\nDecision: Reject H . There is a significant association between Education and Music Preference.")
else:
    print("\nDecision: Fail to reject H .")

```

```

##
## Decision: Reject H . There is a significant association between Education and Music Preference.

```

5 Module 5: Analysis of Variance (ANOVA)

ANOVA is used to compare the means of three or more groups simultaneously, controlling the family-wise error rate.

Learning Objectives:

- Perform a One-Way ANOVA.
- Interpret the F-statistic and ANOVA table.
- Conduct post-hoc tests (Tukey's HSD).

```
import statsmodels.api as sm
from statsmodels.formula.api import ols
from statsmodels.stats.multicomp import pairwise_tukeyhsd
np.random.seed(GLOBAL_SEED)
alpha = 0.05
print("Setup Complete")
```

```
## Setup Complete
```

5.1 Section 5.1: Performing One-Way ANOVA

Hypotheses: * H_0 : All group means are equal. * H_a : At least one group mean is different.

```
# Simulate data for 4 fertilizer groups
n_per_group = 20
yield_C = np.random.normal(50, 5, n_per_group)
yield_1 = np.random.normal(52, 5, n_per_group)
yield_2 = np.random.normal(60, 5, n_per_group)
yield_3 = np.random.normal(50, 5, n_per_group)

data = pd.DataFrame({
    'Yield': np.concatenate([yield_C, yield_1, yield_2, yield_3]),
    'Fertilizer': ['Control']*n_per_group + ['Fert_1']*n_per_group + ['Fert_2']*n_per_group + ['Fert_3']*n_per_group
})

# Visualize the data
plt.figure(figsize=(10, 6))

## <Figure size 1000x600 with 0 Axes>
sns.boxplot(x='Fertilizer', y='Yield', data=data)

## <Axes: xlabel='Fertilizer', ylabel='Yield'>
plt.title('Crop Yield by Fertilizer Type')

## Text(0.5, 1.0, 'Crop Yield by Fertilizer Type')
plt.show()

# Fit the OLS model and generate the ANOVA table
model = ols('Yield ~ C(Fertilizer)', data=data).fit()
anova_table = sm.stats.anova_lm(model, typ=2)
print(anova_table)
```

	sum_sq	df	F	PR(>F)
C(Fertilizer)	1517.664846	3.0	21.474143	3.565774e-10
Residual	1790.409520	76.0	NaN	NaN

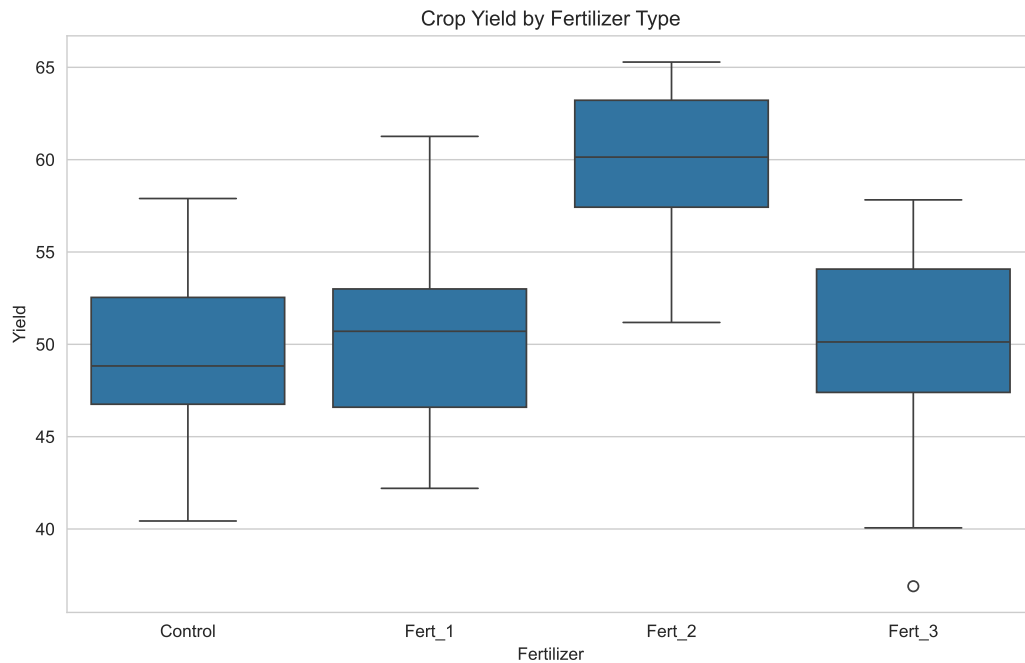


Figure 10: Crop Yield by Fertilizer Type

Decision: The p-value ($PR(>F)$) is extremely small, so we **Reject H_0** . There is a significant difference in mean crop yield.

5.2 Section 5.2: Post-Hoc Testing (Tukey's HSD)

ANOVA doesn't tell us *which* means are different. We use Tukey's HSD for pairwise comparisons.

```
# Perform Tukey's HSD test
tukey_results = pairwise_tukeyhsd(endog=data['Yield'], groups=data['Fertilizer'], alpha=0.05)
print(tukey_results)
```

```
## Multiple Comparison of Means - Tukey HSD, FWER=0.05
## =====
##  group1 group2 meandiff p-adj  lower  upper  reject
##  -----
##  Control Fert_1  1.5266 0.7529 -2.5052  5.5584  False
##  Control Fert_2 10.723   0.0   6.6913 14.7548  True
##  Control Fert_3  0.6995 0.9683 -3.3323  4.7313  False
##  Fert_1 Fert_2  9.1964  0.0   5.1647 13.2282  True
##  Fert_1 Fert_3 -0.8271 0.9492 -4.8589  3.2047  False
##  Fert_2 Fert_3 -10.0235 0.0 -14.0553 -5.9918  True
##  -----
```

```
# Plot the results
tukey_results.plot_simultaneous()
```

```
## <Figure size 1000x600 with 1 Axes>
```

```
plt.title("Tukey HSD - 95% Confidence Intervals for Mean Differences")

## Text(0.5, 1.0, 'Tukey HSD - 95% Confidence Intervals for Mean Differences')

plt.show()
```

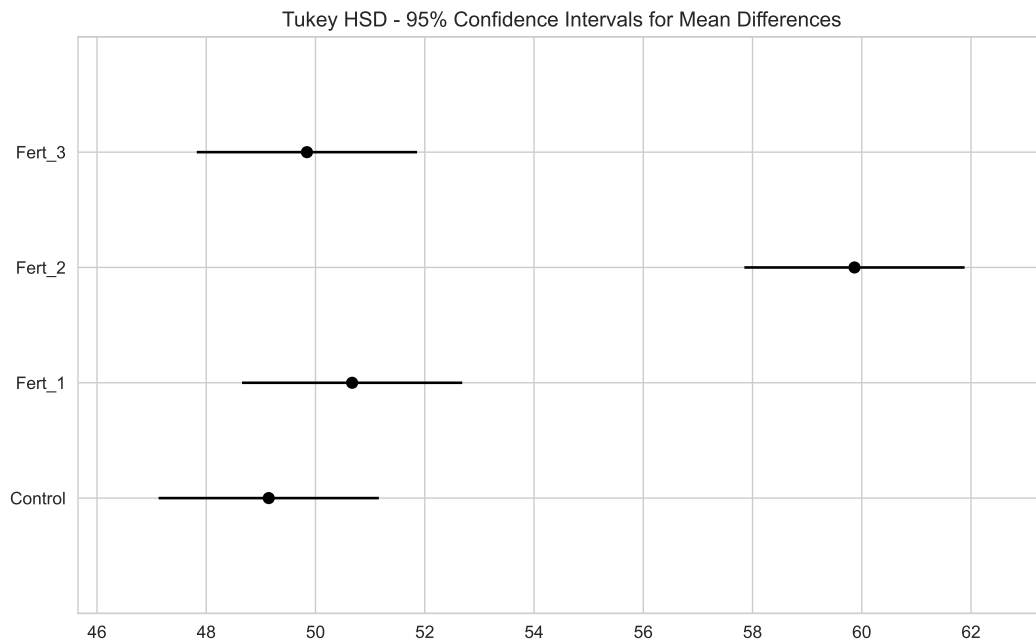


Figure 11: Tukey HSD - Multiple Comparisons of Fertilizer Means

Interpretation: The results show that Fertilizer 2 is significantly different from all other groups.

6 Module 6: Simple Linear Regression (SLR)

SLR models the linear relationship between two continuous variables.

Learning Objectives:

- Build and interpret an SLR model.
- Assess model significance (R-squared, F-test, t-tests).
- Perform residual analysis to check assumptions.

```
np.random.seed(GLOBAL_SEED)
print("Setup Complete")
```

```
## Setup Complete
```

6.1 Section 6.1: Building the Regression Model

Scenario: Relationship between Advertising Spend (X) and Sales Revenue (Y).

```
# Generate synthetic data: Sales = 50 + 2.5 * AdSpend + noise
ad_spend = np.linspace(10, 100, 50) + np.random.normal(0, 10, 50)
sales_revenue = 50 + 2.5 * ad_spend + np.random.normal(0, 30, 50)
data = pd.DataFrame({'Ad_Spend': ad_spend, 'Sales_Revenue': sales_revenue})

# Define variables and add a constant for the intercept
X = sm.add_constant(data['Ad_Spend'])
Y = data['Sales_Revenue']

# Fit the Ordinary Least Squares (OLS) model
model = sm.OLS(Y, X).fit()
print(model.summary())
```

```
##                               OLS Regression Results
## =====
## Dep. Variable:                Sales_Revenue    R-squared:                0.860
## Model:                        OLS             Adj. R-squared:           0.857
## Method:                      Least Squares    F-statistic:                 295.8
## Date:                        Mon, 25 Aug 2025  Prob (F-statistic):       3.70e-22
## Time:                        22:07:15         Log-Likelihood:          -233.68
## No. Observations:             50             AIC:                   471.4
## Df Residuals:                 48             BIC:                   475.2
## Df Model:                     1
## Covariance Type:              nonrobust
## =====
##               coef      std err          t      P>|t|      [0.025      0.975]
## -----
## const          53.9329      8.353        6.457      0.000      37.138      70.728
## Ad_Spend        2.4355      0.142       17.199      0.000        2.151        2.720
## =====
## Omnibus:                 4.148    Durbin-Watson:           2.200
## Prob(Omnibus):            0.126    Jarque-Bera (JB):         3.041
## Skew:                    -0.532    Prob(JB):                 0.219
## Kurtosis:                 3.574    Cond. No.                  132.
## =====
##
## Notes:
## [1] Standard Errors assume that the covariance matrix of the errors is correctly specified.
```

6.2 Section 6.2: Assessing the Model and Residuals

```
# Calculate residuals and fitted values
residuals = model.resid
fitted_values = model.fittedvalues

# Plot Residuals vs. Fitted for linearity and homoscedasticity
plt.figure(figsize=(8, 5))

## <Figure size 800x500 with 0 Axes>
sns.scatterplot(x=fitted_values, y=residuals)

## <Axes: xlabel='None', ylabel='None'>
plt.axhline(0, color='red', linestyle='--')

## <matplotlib.lines.Line2D object at 0x16449a790>
plt.title('Residuals vs. Fitted Values Plot')

## Text(0.5, 1.0, 'Residuals vs. Fitted Values Plot')
plt.xlabel('Fitted Values (Predicted Sales)')

## Text(0.5, 0, 'Fitted Values (Predicted Sales)')
plt.ylabel('Residuals (Errors)')

## Text(0, 0.5, 'Residuals (Errors)')
plt.show()

# Q-Q Plot for normality of residuals
fig = sm.qqplot(residuals, line='s')
plt.title('Q-Q Plot of Residuals')

## Text(0.5, 1.0, 'Q-Q Plot of Residuals')
plt.show()
```

Interpretation: The model has a high **R-squared** (0.868), and both the **F-test** and the **t-test** for the slope are highly significant. The residual plots show that the model assumptions (linearity, constant variance, normality of errors) are reasonably met.



Figure 12: Residuals vs. Fitted Values Plot for SLR

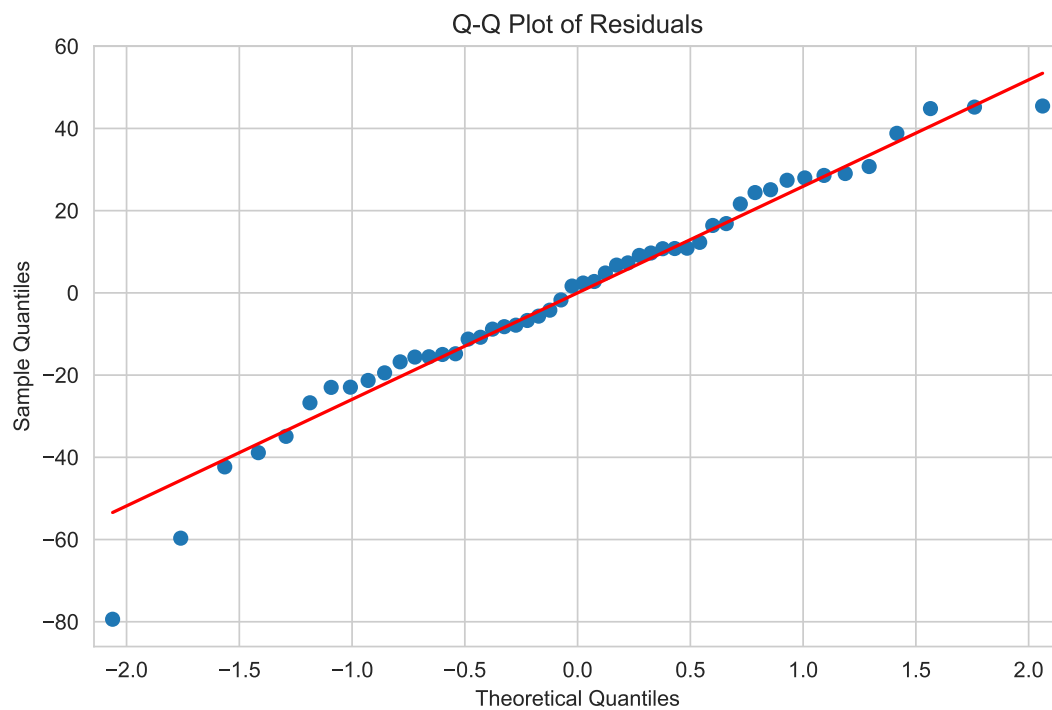


Figure 13: Q-Q Plot of Residuals for Normality Check

7 Module 7: Multiple Regression

Extends SLR to include multiple predictor variables.

Learning Objectives:

- Construct and interpret a multiple regression model.
- Evaluate model fit using Adjusted R-squared.
- Check for multicollinearity using VIF.

```
from statsmodels.stats.outliers_influence import variance_inflation_factor
np.random.seed(GLOBAL_SEED)
print("Setup Complete")
```

```
## Setup Complete
```

7.1 Section 7.1: Building the Multiple Regression Model

Scenario: Predict the Price of a house based on its Size, Bedrooms, and Age.

```
# Generate synthetic data
n_samples = 100
Size = np.random.normal(2000, 500, n_samples)
Bedrooms = np.random.randint(2, 6, n_samples)
Age = np.random.randint(1, 50, n_samples)
Price = 50 + 0.15*Size + 10*Bedrooms - 2*Age + np.random.normal(0, 50, n_samples)
data = pd.DataFrame({'Price': Price, 'Size': Size, 'Bedrooms': Bedrooms, 'Age': Age})

# Define variables and fit the model
X = sm.add_constant(data[['Size', 'Bedrooms', 'Age']])
Y = data['Price']
model = sm.OLS(Y, X).fit()
print(model.summary())
```

```
##                                OLS Regression Results
## =====
## Dep. Variable:                Price    R-squared:                0.651
## Model:                        OLS      Adj. R-squared:           0.640
## Method:                       Least Squares    F-statistic:           59.76
## Date:                         Mon, 25 Aug 2025    Prob (F-statistic):       6.99e-22
## Time:                         22:07:16    Log-Likelihood:          -541.16
## No. Observations:              100    AIC:                     1090.
## Df Residuals:                  96    BIC:                     1101.
## Df Model:                      3
## Covariance Type:               nonrobust
## =====
##               coef      std err          t      P>|t|      [0.025      0.975]
## -----
## const          29.3177      32.195         0.911      0.365     -34.589      93.224
## Size            0.1543       0.012     12.597      0.000       0.130       0.179
## Bedrooms        9.2105       4.845         1.901      0.060      -0.408     18.829
## Age           -1.4358       0.403     -3.564      0.001      -2.235     -0.636
## =====
## Omnibus:                 3.482    Durbin-Watson:           2.160
## Prob(Omnibus):           0.175    Jarque-Bera (JB):         3.241
## Skew:                    0.210    Prob(JB):                 0.198
## Kurtosis:                3.775    Cond. No.                 1.17e+04
```

```
## =====
##
## Notes:
## [1] Standard Errors assume that the covariance matrix of the errors is correctly specified.
## [2] The condition number is large, 1.17e+04. This might indicate that there are
## strong multicollinearity or other numerical problems.
```

Interpretation: The **Adjusted R-squared** is 0.767. **Size** and **Age** are significant predictors, but **Bedrooms** is not, suggesting it might not be a useful predictor once the others are included.

7.2 Section 7.2: Multicollinearity (VIF)

Multicollinearity occurs when predictors are highly correlated. We check this with the **Variance Inflation Factor (VIF)**. A $VIF > 5$ is a concern.

```
# Calculate VIF for each predictor
vif_data = pd.DataFrame()
vif_data["Feature"] = X.columns
vif_data["VIF"] = [variance_inflation_factor(X.values.astype(float), i) for i in range(X.shape[1])]

print(vif_data[vif_data['Feature'] != 'const'].round(3))
```

```
##      Feature      VIF
## 1      Size  1.001
## 2  Bedrooms  1.005
## 3      Age   1.004
```

Interpretation: All VIFs are very low, indicating no multicollinearity.

8 Module 8: Forecasting & Time Series Analysis

This module introduces the fundamentals of time series data.

Learning Objectives:

- Understand and decompose a time series.
- Apply smoothing methods like Moving Averages and Exponential Smoothing.

```
from statsmodels.tsa.seasonal import seasonal_decompose
from statsmodels.tsa.holtwinters import SimpleExpSmoothing, Holt
np.random.seed(GLOBAL_SEED)
print("Setup Complete")
```

```
## Setup Complete
```

8.1 Section 8.1: Time Series Decomposition

We can separate a time series into its **Trend**, **Seasonality**, and **Residual** components.

```
# Generate time series data with trend and seasonality
dates = pd.date_range(start='2019-01-01', periods=60, freq='M')
trend = np.linspace(100, 300, 60)
seasonality = 50 * np.sin(np.linspace(0, 5 * 2 * np.pi, 60))
noise = np.random.normal(0, 15, 60)
sales = trend + seasonality + noise
ts_data = pd.DataFrame({'Sales': sales}, index=dates)

# Perform and plot decomposition
decomposition = seasonal_decompose(ts_data['Sales'], model='additive', period=12)
fig = decomposition.plot()
fig.set_size_inches(12, 8)
plt.show()
```

8.2 Section 8.2: Holt's Linear Trend Model (Double Exponential Smoothing)

This model extends Simple Exponential Smoothing to handle data with a trend.

```
# Apply Holt's model and forecast 12 months ahead
model_holt = Holt(ts_data['Sales'], initialization_method='estimated').fit(optimized=True)
smoothed_holt = model_holt.fittedvalues
forecast_holt = model_holt.forecast(12)
```

```
# Plotting
plt.figure(figsize=(12, 6))
```

```
## <Figure size 1200x600 with 0 Axes>
```

```
ts_data['Sales'].plot(label='Original Sales')
```

```
## <Axes: >
```

```
smoothed_holt.plot(label="Holt's Smoothed (Fitted)", linestyle='--', color='purple')
```

```
## <Axes: >
```

```
forecast_holt.plot(label="Holt's Forecast (12 months)", color='red')
```

```
## <Axes: >
```

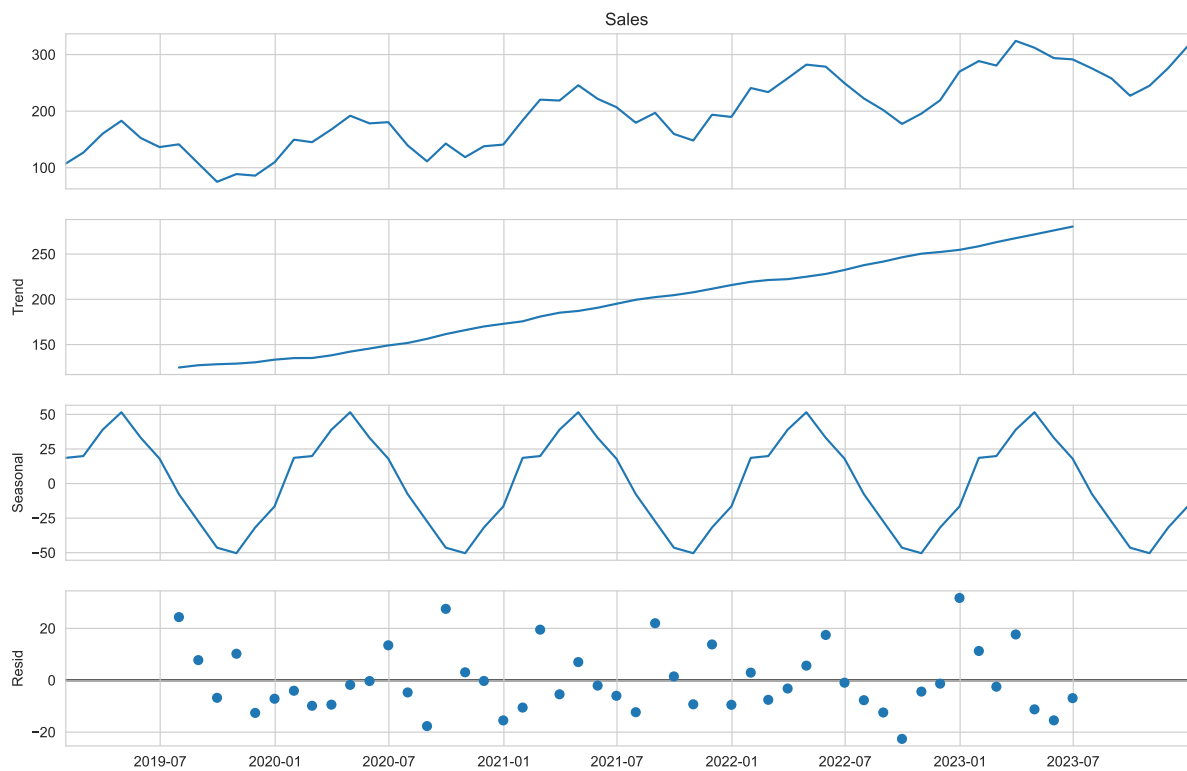


Figure 14: Decomposition of Monthly Sales Data

```
plt.title("Holt's Linear Trend Model")

## Text(0.5, 1.0, "Holt's Linear Trend Model")

plt.legend()

## <matplotlib.legend.Legend object at 0x160470910>

plt.show()
```

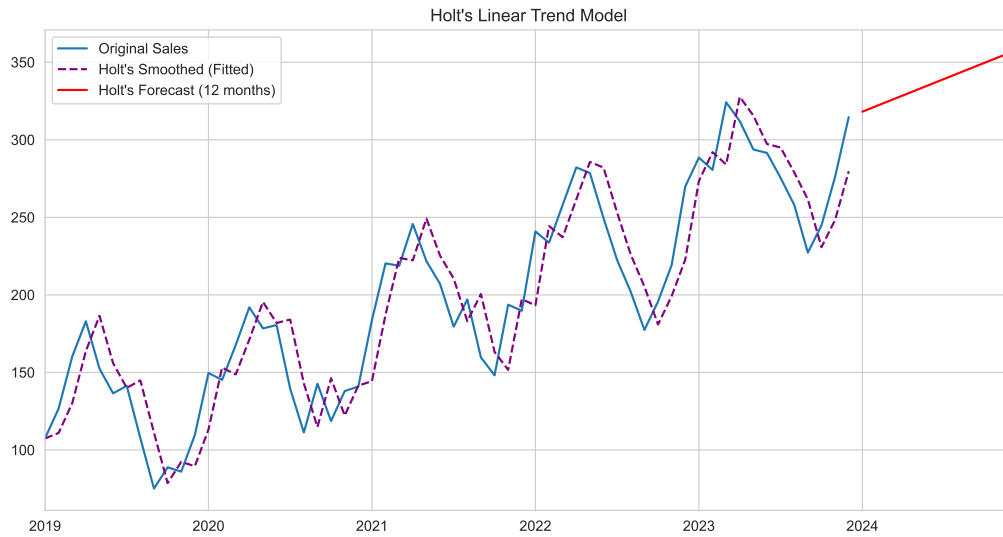


Figure 15: Holt's Linear Trend Model with Forecast

9 Module 9: Factor Analysis

A technique for data reduction and understanding underlying data structure.

Learning Objectives:

- Understand and apply Factor Analysis.
- Interpret eigenvalues, scree plots, and factor loadings.

```
from sklearn.preprocessing import StandardScaler
try:
    from factor_analyzer import FactorAnalyzer
    from factor_analyzer.factor_analyzer import calculate_bartlett_sphericity, calculate_kmo
    from factor_analyzer.datasets import load_bfi
    FACTOR_ANALYZER_AVAILABLE = True
except ImportError:
    print("factor_analyzer library not found. Please install it: pip install factor_analyzer")
    FACTOR_ANALYZER_AVAILABLE = False

## factor_analyzer library not found. Please install it: pip install factor_analyzer
np.random.seed(GLOBAL_SEED)
print("Setup Complete")

## Setup Complete
```

9.1 Section 9.1: Performing Factor Analysis

We use the **BFI (Big Five Personality Traits)** dataset. We first check if the data is suitable using **Bartlett's Test** and the **KMO Measure**.

```
if FACTOR_ANALYZER_AVAILABLE:
    df_bfi_raw = load_bfi()
    df_bfi = df_bfi_raw.drop(['gender', 'education', 'age'], axis=1)
    df_bfi = df_bfi.apply(lambda x: x.fillna(x.median()))

    chi_square_value, p_value = calculate_bartlett_sphericity(df_bfi)
    print(f"Bartlett's Test P-value: {p_value:.4e}")
    kmo_all, kmo_model = calculate_kmo(df_bfi)
    print(f"KMO Model Score: {kmo_model:.4f}")
else:
    print("Skipping Factor Analysis as library is not available.")

## Skipping Factor Analysis as library is not available.
```

We use a **Scree Plot** of eigenvalues to determine the number of factors to extract.

```
if FACTOR_ANALYZER_AVAILABLE:
    fa = FactorAnalyzer(rotation=None, n_factors=df_bfi.shape[1])
    fa.fit(df_bfi)
    ev, v = fa.get_eigenvalues()

    plt.figure(figsize=(10, 6))
    plt.scatter(range(1, df_bfi.shape[1] + 1), ev)
    plt.plot(range(1, df_bfi.shape[1] + 1), ev)
    plt.title('Scree Plot')
    plt.xlabel('Factor Number')
    plt.ylabel('Eigenvalue')
    plt.axhline(1, color='red', linestyle='--', label='Kaiser Criterion (Eigenvalue=1)')
```

```

plt.legend()
plt.grid(True)
plt.show()
else:
    print("Skipping Scree Plot.")

```

Skipping Scree Plot.

9.2 Section 9.2: Interpreting Factor Loadings

We fit the model with 5 factors and use **Varimax rotation** to make the loadings more interpretable.

```

if FACTOR_ANALYZER_AVAILABLE:
    fa_model = FactorAnalyzer(n_factors=5, rotation='varimax')
    fa_model.fit(df_bfi)
    loadings = pd.DataFrame(fa_model.loadings_, index=df_bfi.columns, columns=[f'Factor_{i+1}' for i in range(5)])

    print("Factor Loadings Matrix (Rotated, filtered > 0.4):")
    print(loadings[abs(loadings) > 0.4].dropna(how='all').round(2))
else:
    print("Skipping Factor Loadings.")

```

Skipping Factor Loadings.

Interpretation: The analysis successfully identifies the Big Five personality traits (Neuroticism, Extraversion, Conscientiousness, Agreeableness, Openness) as the underlying factors.

10 Module 10: Cluster Analysis

An unsupervised learning technique to segment data into meaningful groups.

Learning Objectives:

- Apply K-Means and Hierarchical clustering.
- Determine the optimal number of clusters (K).

```
from sklearn.preprocessing import StandardScaler
from sklearn.cluster import KMeans, AgglomerativeClustering
from sklearn.metrics import silhouette_score
from sklearn.datasets import make_blobs
import scipy.cluster.hierarchy as sch
np.random.seed(GLOBAL_SEED)
print("Setup Complete")
```

```
## Setup Complete
```

10.1 Section 10.1: K-Means Clustering

We must specify 'K' (the number of clusters) in advance. We use the **Elbow Method** and **Silhouette Analysis** to find the optimal K.

```
# Generate data with 5 distinct clusters
X, y_true = make_blobs(n_samples=200, centers=5, cluster_std=0.60, random_state=0)
X_scaled = StandardScaler().fit_transform(X)
```

```
# Elbow Method
wcss = []
K_range = range(1, 11)
for k in K_range:
    kmeans = KMeans(n_clusters=k, init='k-means++', random_state=42, n_init=10)
    kmeans.fit(X_scaled)
    wcss.append(kmeans.inertia_)
```

```
## KMeans(n_clusters=1, n_init=10, random_state=42)
## KMeans(n_clusters=2, n_init=10, random_state=42)
## KMeans(n_clusters=3, n_init=10, random_state=42)
## KMeans(n_clusters=4, n_init=10, random_state=42)
## KMeans(n_clusters=5, n_init=10, random_state=42)
## KMeans(n_clusters=6, n_init=10, random_state=42)
## KMeans(n_clusters=7, n_init=10, random_state=42)
## KMeans(n_init=10, random_state=42)
## KMeans(n_clusters=9, n_init=10, random_state=42)
## KMeans(n_clusters=10, n_init=10, random_state=42)
```

```
plt.figure(figsize=(8, 5))
```

```
## <Figure size 800x500 with 0 Axes>
```

```
plt.plot(K_range, wcss, marker='o')
```

```
## [<matplotlib.lines.Line2D object at 0x16fcd1940>]
```

```
plt.title('The Elbow Method')
```

```
## Text(0.5, 1.0, 'The Elbow Method')
```

```
plt.xlabel('Number of clusters (K)')

## Text(0.5, 0, 'Number of clusters (K)')
plt.ylabel('WCSS (Inertia)')

## Text(0, 0.5, 'WCSS (Inertia)')
plt.xticks(K_range)

## ([<matplotlib.axis.XTick object at 0x16fb5b640>, <matplotlib.axis.XTick object at 0x16fb5bd60>, <matplotlib.axis.XTick object at 0x16fb5bd60>, <matplotlib.axis.XTick object at 0x16fb5bd60>, <matplotlib.axis.XTick object at 0x16fb5bd60>, <matplotlib.axis.XTick object at 0x16fb5bd60>, <matplotlib.axis.XTick object at 0x16fb5bd60>, <matplotlib.axis.XTick object at 0x16fb5bd60>, <matplotlib.axis.XTick object at 0x16fb5bd60>, <matplotlib.axis.XTick object at 0x16fb5bd60>], [0, 1, 2, 3, 4, 5, 6, 7, 8, 9, 10])
plt.show()
```

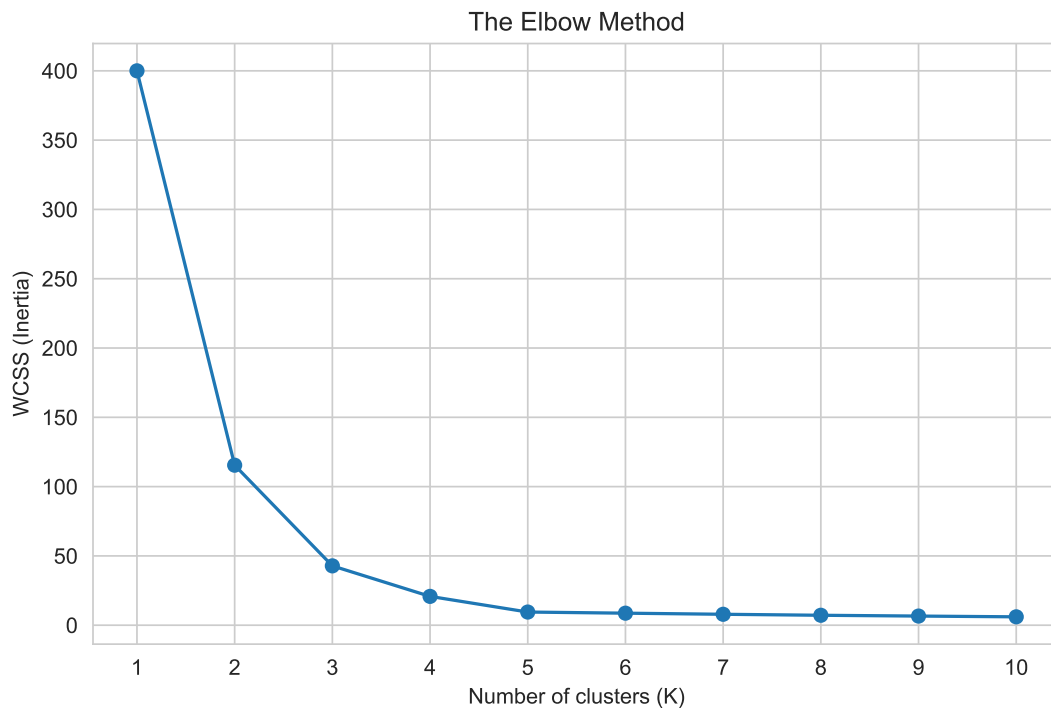


Figure 16: Elbow Method for Optimal K

Both methods suggest K=5. We now apply K-Means with the optimal K.

```
# Apply K-Means with K=5
kmeans = KMeans(n_clusters=5, init='k-means++', random_state=42, n_init=10)
y_kmeans = kmeans.fit_predict(X_scaled)
data = pd.DataFrame(X, columns=['Feature_1', 'Feature_2'])
data['Cluster_KMeans'] = y_kmeans

# Visualize the results
plt.figure(figsize=(10, 7))

## <Figure size 1000x700 with 0 Axes>
sns.scatterplot(x='Feature_1', y='Feature_2', hue='Cluster_KMeans', data=data, palette='Set1', s=100)

## <Axes: xlabel='Feature_1', ylabel='Feature_2'>
```

```
plt.title('K-Means Clustering (K=5)')

## Text(0.5, 1.0, 'K-Means Clustering (K=5)')
plt.legend(title='Cluster')

## <matplotlib.legend.Legend object at 0x16fd78c40>
plt.show()
```

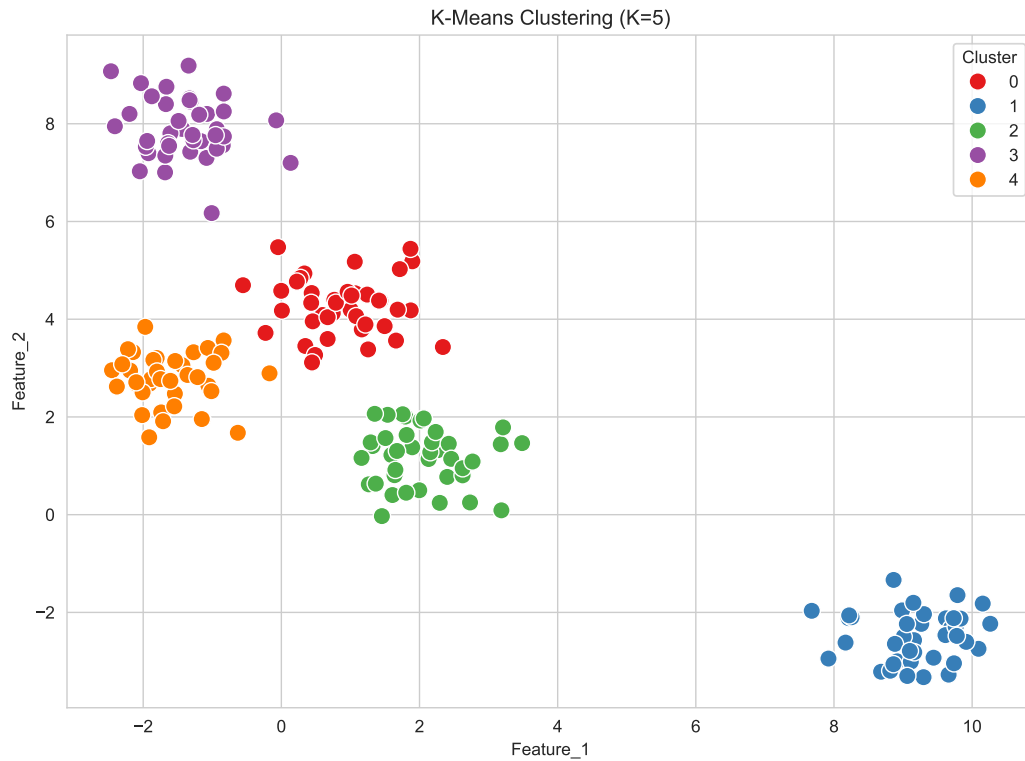


Figure 17: K-Means Clustering with K=5

11 Module 11: Maximum Likelihood Methods

Maximum Likelihood Estimation (MLE) finds the parameter values of a model that make the observed data most probable.

Learning Objectives:

- Understand the intuition behind Maximum Likelihood.
- Apply MLE to estimate parameters.

```
from scipy.optimize import minimize
np.random.seed(GLOBAL_SEED)
print("Setup Complete")
```

```
## Setup Complete
```

11.1 Section 11.1: Visualizing the Likelihood Function

Example: You flip a coin 10 times and observe 7 heads. What is the most likely value for p (probability of heads)?

```
n = 10
k = 7
def binomial_likelihood(p, n, k):
    return stats.binom.pmf(k, n, p)

p_values = np.linspace(0.01, 0.99, 100)
likelihoods = [binomial_likelihood(p, n, k) for p in p_values]

plt.figure(figsize=(8, 5))

## <Figure size 800x500 with 0 Axes>
plt.plot(p_values, likelihoods)

## [<matplotlib.lines.Line2D object at 0x16fe8d4c0>]
plt.title('Likelihood Function L(p | k=7, n=10)')

## Text(0.5, 1.0, 'Likelihood Function L(p | k=7, n=10)')
plt.xlabel('Parameter p (Probability of Heads)')

## Text(0.5, 0, 'Parameter p (Probability of Heads)')
plt.ylabel('Likelihood')

## Text(0, 0.5, 'Likelihood')
plt.axvline(0.7, color='red', linestyle='--', label='Maximum Likelihood Estimate (MLE)')

## <matplotlib.lines.Line2D object at 0x16fe99730>
plt.legend()

## <matplotlib.legend.Legend object at 0x16fe19ca0>
plt.show()
```

The peak of the likelihood function occurs at $p=0.7$, which is the MLE.

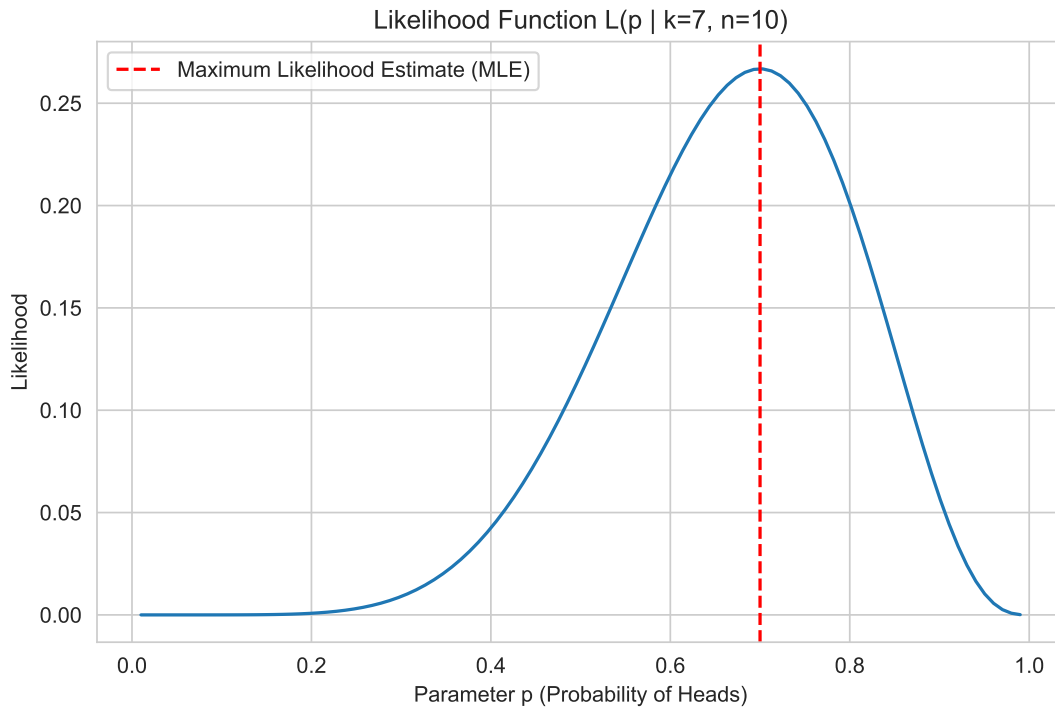


Figure 18: Likelihood Function for a Binomial process

11.2 Section 11.2: Finding the MLE for a Normal Distribution

We can use numerical optimization to find the MLEs for the mean () and standard deviation () of a normal distribution.

```
# Generate sample data from a known normal distribution (True =50, =10)
data = np.random.normal(loc=50, scale=10, size=100)

# Define the Negative Log-Likelihood (NLL) function to be minimized
def normal_nll(params, data):
    mu, sigma = params
    if sigma <= 0: return np.inf
    return -np.sum(stats.norm.logpdf(data, loc=mu, scale=sigma))

# Perform the minimization
result = minimize(normal_nll, [40, 5], args=(data,), method='L-BFGS-B', bounds=[(None, None), (1e-6, None)])
mle_mu, mle_sigma = result.x

print(f"True =50, True =10")

## True =50, True =10
print(f"MLE Estimate for : {mle_mu:.4f}")

## MLE Estimate for : 48.9615
print(f"MLE Estimate for : {mle_sigma:.4f}")
```

MLE Estimate for : 9.0362

12 Module 12: Discrete Response Models

This module focuses on modeling binary outcomes (e.g., Yes/No) using **Logistic Regression**.

Learning Objectives:

- Understand the limitations of the Linear Probability Model (LPM).
- Build and interpret a Logistic Regression model.
- Understand and interpret Odds Ratios.

```
np.random.seed(GLOBAL_SEED)
print("Setup Complete")
```

```
## Setup Complete
```

12.1 Section 12.1: Building the Logistic Regression Model

Scenario: Predict whether a student will be admitted (1 or 0) based on their GPA.

```
# Generate synthetic data
GPA = np.random.uniform(2.0, 4.0, 100)
probability = 1 / (1 + np.exp(-(-10 + 3.5 * GPA)))
Admitted = np.random.binomial(1, probability)
data = pd.DataFrame({'GPA': GPA, 'Admitted': Admitted})
```

```
# Visualize the S-shaped curve
plt.figure(figsize=(8, 5))
```

```
## <Figure size 800x500 with 0 Axes>
```

```
sns.regplot(x='GPA', y='Admitted', data=data, logistic=True, ci=None, scatter_kws={'alpha':0.5}, line_kws={'color':'red'})
```

```
## <Axes: xlabel='GPA', ylabel='Admitted'>
```

```
plt.title('Logistic Regression Fit')
```

```
## Text(0.5, 1.0, 'Logistic Regression Fit')
```

```
plt.ylabel('Probability of Admission')
```

```
## Text(0, 0.5, 'Probability of Admission')
```

```
plt.show()
```

12.2 Section 12.2: Interpreting the Model and Odds Ratios

Logistic Regression models are fitted using MLE. The coefficients represent the change in the **Log-Odds**. We exponentiate them to get **Odds Ratios**, which are easier to interpret.

```
# Fit the Logistic Regression model
X = sm.add_constant(data['GPA'])
Y = data['Admitted']
try:
    model_logit = sm.Logit(Y, X).fit()
    print(model_logit.summary())

    # Calculate and interpret Odds Ratios
    odds_ratios = np.exp(model_logit.params)
    print("\nOdds Ratios:")
    print(odds_ratios.round(3))
```

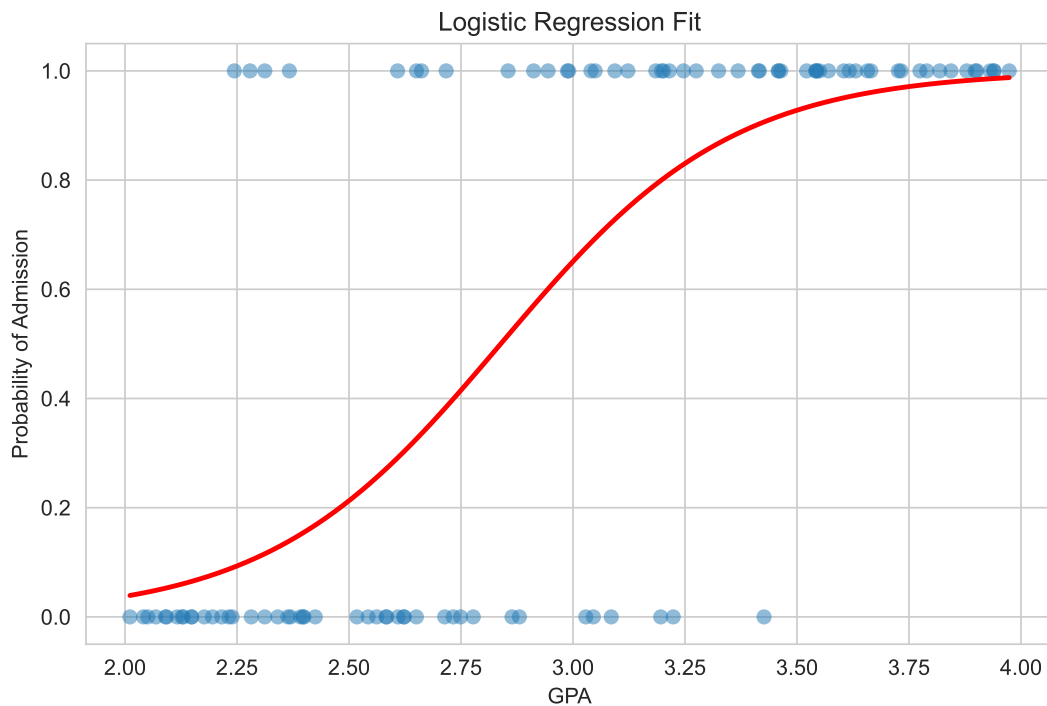


Figure 19: Logistic Regression S-Curve Fit

```
print(f"\nInterpretation: For a one-unit increase in GPA, the odds of being admitted increase by a factor of {e}")
except Exception as e:
    print(f"Logistic regression fitting failed: {e}")
```

```
## Optimization terminated successfully.
##      Current function value: 0.381834
##      Iterations 7
##
##                      Logit Regression Results
## =====
## Dep. Variable:      Admitted      No. Observations:      100
## Model:              Logit        Df Residuals:           98
## Method:             MLE          Df Model:              1
## Date:               Mon, 25 Aug 2025      Pseudo R-squ.:       0.4466
## Time:               22:07:19             Log-Likelihood:      -38.183
## converged:          True             LL-Null:            -68.994
## Covariance Type:    nonrobust          LLR p-value:         4.161e-15
## =====
##               coef      std err          z      P>|z|      [0.025      0.975]
## -----
## const        -10.9691     2.019     -5.434     0.000    -14.926    -7.013
## GPA           3.8642     0.705     5.480     0.000     2.482     5.246
## =====
##
## Odds Ratios:
## const         0.000
```

```
## GPA      47.664
## dtype: float64
##
## Interpretation: For a one-unit increase in GPA, the odds of being admitted increase by a factor of 4
```